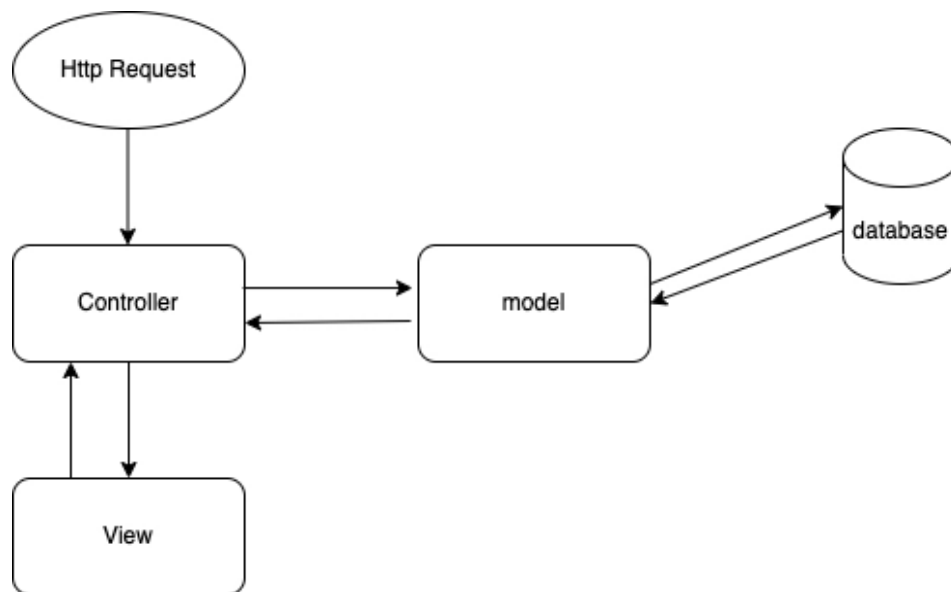




MVC Core Notes

- IoC
- DI
- Lifecycles

are core to MVC



Create a Spring Boot Project (Maven)

You can generate it using:

- Spring Initializr (web UI), or
- CLI / IDE (IntelliJ, VS Code)

Basic structure (pom.xml present):

Add JasperReports Dependency

In your `pom.xml`, add:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
          xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
          xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
                                http://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>com.example</groupId>
    <artifactId>springboot-jasper-app</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>springboot-jasper-app</name>
    <description>Spring Boot with JasperReports and JSTL</description>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.2.5</version>
        <relativePath/>
    </parent>

    <properties>
        <java.version>17</java.version>
        <jasper.version>6.21.0</jasper.version>
    </properties>

    <dependencies>

        <!-- Spring Boot Web -->
        <dependency>
```

```

        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <!-- JSP support (Tomcat Jasper) -->
    <dependency>
        <groupId>org.apache.tomcat.embed</groupId>
        <artifactId>tomcat-embed-jasper</artifactId>
    </dependency>

    <!-- JSTL -->
    <dependency>
        <groupId>jakarta.servlet.jsp.jstl</groupId>
        <artifactId>jakarta.servlet.jsp.jstl-api</artifac
tId>
    </dependency>

    <dependency>
        <groupId>org.glassfish.web</groupId>
        <artifactId>jakarta.servlet.jsp.jstl</artifactId>
    </dependency>

    <!-- Devtools (optional) -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-devtools</artifactId>
        <scope>runtime</scope>
    </dependency>

    <!-- Lombok (optional) -->
    <dependency>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
        <optional>true</optional>
    </dependency>

```

```

    <!-- Test -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>

</dependencies>

<build>
    <plugins>

        <!-- Spring Boot Maven Plugin -->
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
            <configuration>
                <mainClass>com.example.demo.DemoApplication</mainClass>
            </configuration>
        </plugin>

    </plugins>
</build>

</project>

```

Spring Boot uses the Maven plugin to run and package:

```

<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
    </plugins>
</build>

```

```
</plugins>  
</build>
```

Core Spring Boot

- `spring-boot-starter-web`

Provides everything needed to build a web app:

- Embedded Tomcat server
- Spring MVC (controllers, REST APIs)
- JSON handling (Jackson)

JSP + View Layer

- `tomcat-embed-jasper`

Enables JSP compilation inside the embedded Tomcat server.

Without this, JSP files won't render.

- `jakarta.servlet.jsp.jstl-api`

Defines JSTL tags (like `<c:forEach>`, `<c:if>`).

This is just the API (interfaces).

- `jakarta.servlet.jsp.jstl` **(GlassFish impl)**

Actual implementation of JSTL so the tags work at runtime.

You need both API + implementation.

Reporting (Jasper)

- `jasperreports`

Core reporting engine:

- Compiles `.jrxml` → `.jasper`
- Fills reports with data
- Exports to PDF, Excel, HTML

- `jasperreports-fonts` (optional)

Provides font support for exports (especially PDFs).

Helps avoid missing font / encoding issues.

Developer Convenience

- `spring-boot-devtools` (runtime)

Auto-restarts app on code changes.

Speeds up development, not used in production.

- `lombok` (optional)

Reduces boilerplate:

- Generates getters/setters
- Constructors, builders

Requires IDE plugin.

Testing

- `spring-boot-starter-test`

Includes:

- JUnit
- Mockito
- Spring test utilities

Used for writing unit/integration tests.

Build Plugin

- `spring-boot-maven-plugin`

Lets you:

- Run app: `mvn spring-boot:run`
- Package executable JAR

- Handle dependency bundling

Setup project env II

- JSP java server page (view)



handles the presentation layer. Java code executed on the server side to generate HTML sent to client's web browser

In the spring boot project, there is a problem with web services and servers. What dependency might not be added to the pom.xml file?

The Spring Web dependency is a part of the Spring Framework and is used to build web applications in Spring Web. It provides several key components for building web applications, such as Spring's Web MVC framework for handling requests and generating responses, support for handling HTTP requests and responses, including requests and query parameters.

```
spring:
  mvc:
    view:
      prefix: /WEB-INF/views/
      suffix: .jsp

  servlet:
    multipart:
      max-file-size: 10MB
      max-request-size: 10MB

server:
  port: 8080
```

```

springboot-jasper-app/
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   ├── com/example/demo/
│   │   │   │   ├── controller/
│   │   │   │   │   └── ReportController.java
│   │   │   │   ├── service/
│   │   │   │   │   ├── ReportService.java
│   │   │   │   └── DemoApplication.java
│   │   ├── resources/
│   │   │   ├── application.yml
│   │   │   └── reports/
│   │   │       └── sample.jrxml
│   │   └── webapp/
│   │       ├── WEB-INF/
│   │       │   └── views/
│   │       │       ├── home.jsp
│   │       │       └── report.jsp
│   └── pom.xml

```

Components

1. domain
2. repository : handles storage and retrieval of data, acting as a bridge between the app and databases.
3. service: handles specific actions or operations the user requests like creating an account.

Home → Registration → Success

4. Web layer: handles and manages the interaction between the user's web browser and app, including handling user input and generating responses. Services makes sure domain is called and domain calls repositories.

Flow

Web → registration service → domain objects → repository

Create web and service layer

and understand how to configure multiple controllers

Service layer → web layer → controllers

```
package com.example.website.web;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;

@Controller
public class HomeController {

    @RequestMapping("/home")
    public String getHomePage() {
        return "home";
    }
}
```

```
package com.example.website.web;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;


```

```

@Controller
public class SignupController {

    @RequestMapping("/signup")
    public String getSignupPage() {
        return "signup";
    }
}

```

```

<!-- /WEB-INF/views/home.jsp -->
<%@ page contentType="text/html; charset=UTF-8" language="java" %>
<!DOCTYPE html>
<html>
<head>
    <title>Home</title>
</head>
<body>
    <h1>Welcome to Home Page</h1>
    <a href="signup">Go to Signup</a>
</body>
</html>

```

```

<!-- /WEB-INF/views/signup.jsp -->
<%@ page contentType="text/html; charset=UTF-8" language="java" %>
<!DOCTYPE html>
<html>
<head>
    <title>Signup</title>
</head>

```

```
</body>  
</html>
```

`@RequestMapping` is a Spring MVC annotation used to map HTTP requests to handler methods (or classes) in a controller.

```
@RequestMapping("/home")  
public String getHomePage() {  
    return "home";  
}
```

Domain and domain objects

1. What is a Domain?

A **domain** is the real-world context your application models.

Examples:

- E-commerce app → domain = orders, users, payments, products
- Job portal → domain = jobs, applicants, resumes
- Banking app → domain = accounts, transactions, loans

So the domain is basically:

| "What your system is about"

2. What are Domain Objects?

Domain objects are Java classes that represent real-world entities in your domain.

They usually:

- Hold data (fields)

- Represent business concepts
- Sometimes include business logic

```
public class StudentUser extends User {

    private String college;

    public String getCollege() {
        return college;
    }

    public void setCollege(String college) {
        this.college = college;
    }

    @Override
    public boolean createUser(String name, String gender, String location, String college) {
        this.name = name;
        this.gender = gender;
        this.location = location;
        this.college = college;
        return true;
    }

    @Override
    public Integer saveUser() {
        // implementation not fully visible in image
        return 1;
    }
}
```

```
website/
└─ src/
```



```
|— pom.xml
|— README.md
```

We use a service layer to connect the web request to the rest of the application. The controller (web layer) should remain thin: it receives the HTTP request, performs minimal validation/binding, and delegates to the service. The service coordinates domain objects, applies business rules, and interacts with repositories. This separation keeps responsibilities clear, improves testability, and avoids putting business logic inside controllers or persistence code.

In a signup flow, the controller maps `/signup` and binds form data to a request object. It then calls something like `userService.register(request)`. The service validates inputs (e.g., email format, password rules, uniqueness), creates or updates domain objects, and persists them via a repository. It also handles transactions and may publish domain events (e.g., "UserRegistered") for side effects like sending emails.

The domain layer represents the core business model. Domain objects (entities, value objects) should capture the real concepts of the system and enforce invariants. For a registration form, a `User` (or `StudentUser`) is not just a data container; it should express valid states and behaviors. Typical characteristics:

- Encapsulation of state with controlled mutation. Fields like `email`, `password`, `location`, `college` should not be freely set from anywhere; changes happen through methods that enforce rules.
- Invariants are protected. Examples: email must be valid and unique (uniqueness often checked via service/repository), password must meet policy, required fields cannot be null.
- Behavior alongside data. Instead of setters everywhere, provide intention-revealing methods such as `register(...)`, `changePassword(...)`, or `updateProfile(...)`.
- Independence from frameworks where possible. Domain objects should not depend on web or persistence APIs; they model the business, not the transport or storage.
- Clear identity. Entities have identity (e.g., `id` or unique email); value objects (like an `Email` type) are immutable and compared by value.

- Consistency boundaries. A single domain object should remain internally consistent after any operation.

Applied to registration, the service orchestrates creation while the domain enforces rules:

- Controller: receives form → calls service.
- Service: validates input, checks uniqueness via repository, constructs the domain object, invokes domain behavior, saves it.
- Domain: ensures a valid user can be created and maintains invariants.
- Repository: persists the domain object.

```
package com.example.website.service;

import com.example.website.domain.User;

public interface UserService {

    public User getUser();

    public boolean signUp(String name, String gender, String
location, String college);
}
```

```
package com.example.website.service;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import com.example.website.domain.User;

@Service
public class StudenUserService implements UserService {
```

```

@Autowired
User studentUser;

@Override
public User getUser() {
    return studentUser;
}

@Override
public boolean signUp(String name, String gender, String
location, String college) {
    boolean isStudnetCreated =
        studentUser.createUser(name, gender, locatio
n, college);

    studentUser.saveUser();

    return isStudnetCreated;
}
}

```

When developing big projects, it is a mandatory practice to use the Interface and its Implementation class when writing service and domain classes to - Promote clear separation of interfaces.

- Promote modularity and flexibility in design.
- Allow for easier testing and maintenance of code.

Model

```

package com.example.website.web;

import org.springframework.beans.factory.annotation.Autowired;

```

```

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.RequestMapping;

import com.example.website.domain.User;
import com.example.website.service.UserService;

@Controller
public class SignUpController {

    @Autowired
    UserService userService;

    @RequestMapping("/signup")
    public String getSignUpPage(Model uiModel) {
        User user = userService.getUser();
        uiModel.addAttribute("user", user);
        return "signup";
    }
}

```

We have now successfully **connected our service layer to the view layer**.

What this means in your flow:

- The **controller** receives the request (`/signup`)
- It calls the **service layer** (`userService.getUser()`)
- The service returns a **domain object** (`User`)
- The controller adds this object to the **Model**
- The **JSP view** (`signup.jsp`) can now access and display that data

So effectively:

```
Request → Controller → Service → Domain Object → Model → View (JSP)
```

This connection is important because:

- The view should not directly access business logic
- The controller acts as a bridge between UI and backend
- The service handles all business operations
- The domain object carries the actual data

At this point, your application is properly layered:

- Web layer → handles requests
- Service layer → handles logic
- Domain layer → holds data + behavior
- View layer → renders UI

This is the core MVC flow working correctly.

Form handling using JSP

- look at student service and make sure it has Component annotation

Now we can communicate downstream to JSP using our UI Model and set forms based on it, understand JSP benefits and how quickly we can build the forms.

```
<%@ taglib prefix="form" uri="http://www.springframework.org/
tags/form" %>
<html>
<head>
  <title>Sign Up</title>
</head>
<body>

<h1>sign up Page</h1>

<form:form action="registerUser" modelAttribute="user">
```

```

name:
<form:input path="name"/>
<br/>
<br/>

Gender:
<br/>
Male <form:radiobutton path="gender" value="male"/>
Female <form:radiobutton path="gender" value="female"/>
<br/>
<br/>

Location:
<form:select path="location">
    <form:option value="India"></form:option>
    <form:option value="NRI"></form:option>
</form:select>
<br/>
<br/>

College:
<form:select path="college">
    <form:option value="test123"></form:option>
    <form:option value="testCollege"></form:option>
</form:select>
<br/>
<br/>

<input type="submit"/>

</form:form>

</body>
</html>

```

next we will see how to actually register the user

```

@RequestMapping("/signUp")
public String getSignUpPage(Model uiModel) {
    User user = userService.getUser();
    uiModel.addAttribute("user", user);
    return "signup";
}

@RequestMapping("/registerUser")
public String createUser(@ModelAttribute(value = "user") StudentUser studentUser) {

    if (userService.signUp(
        studentUser.getName(),
        studentUser.getGender(),
        studentUser.getLocation(),
        studentUser.getCollege())) {

        return "welcome";
    }

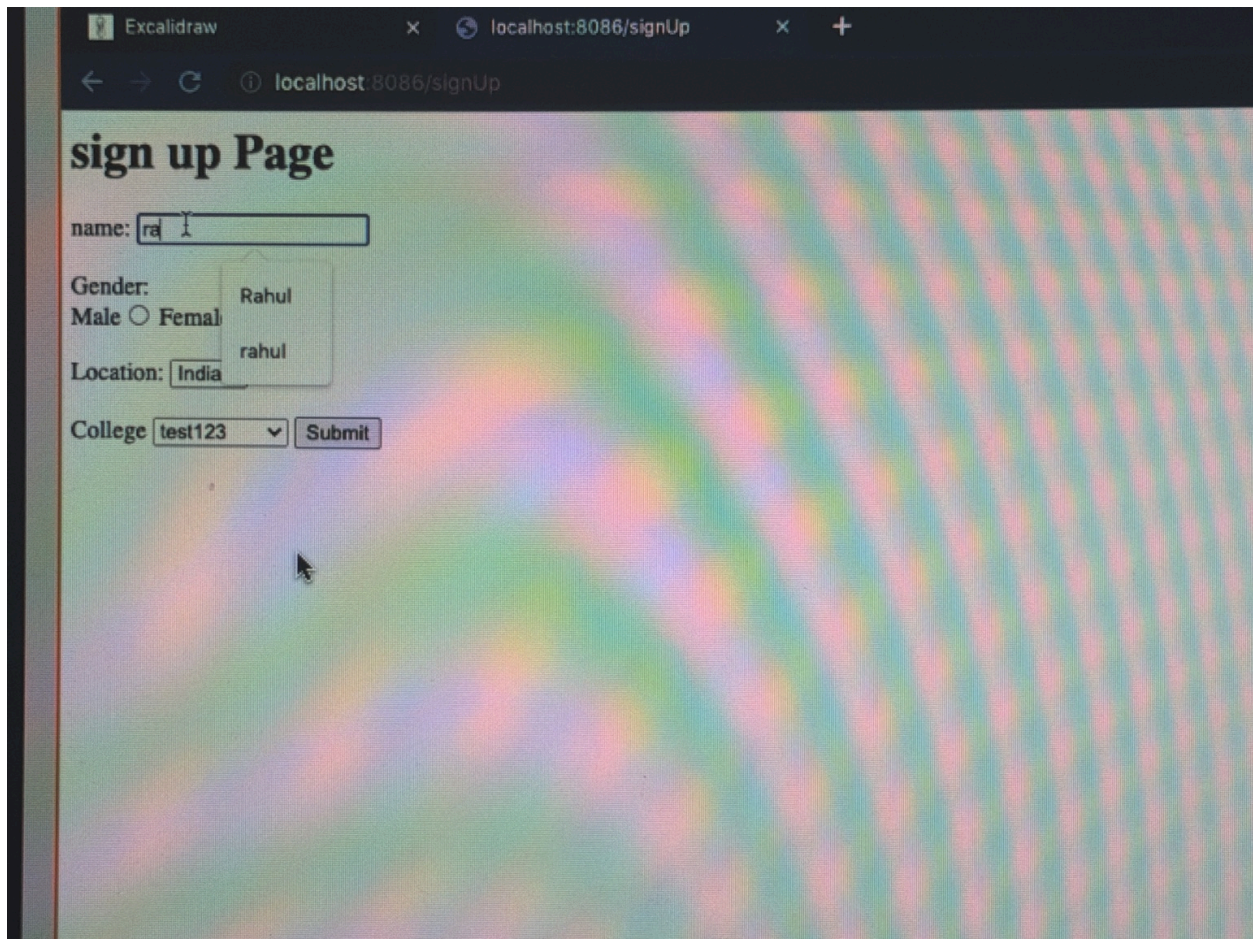
    return "signup";
}

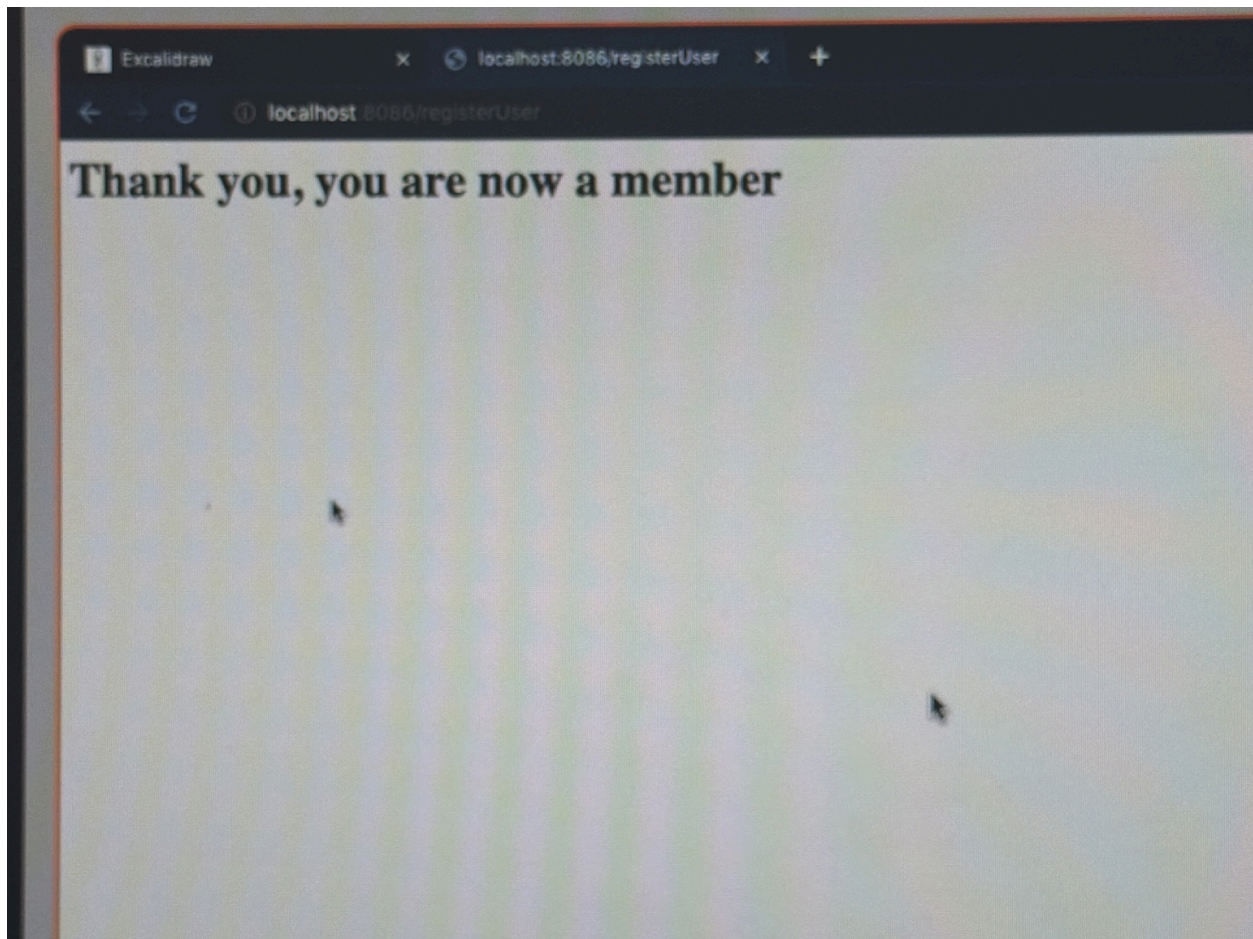
```



The `ModelAttribute` annotation is used to bind JSP page details to a domain object. This means that the values from the JSP form can be automatically populated into a Java object when the form is submitted. The Java object name and model attribute is stored in the model map and can be accessed in the controller method handling the form submission.

and a H1 in JSP for welcome.jsp that says Thank you





Persistence layer I (DAO)

We need to save our objects somewhere, our repository that connects to a DB or an array.

Sits on top of Domain.

Web → service → domain → Repo

Repository layer has classes called DAOs → stands for data access objects.

To write our own DAOs:

- generics
- optional

using these concepts

```
package com.example.website.repository;

import java.util.Optional;

public interface DAO<T> {

    public Optional<T> get(Integer id);

    public int save(T t);
}
```

```
import org.springframework.stereotype.Repository;

import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

@Repository
public class StudentUserDAO implements DAO<StudentUser> {

    private List<StudentUser> studentUserList = new ArrayList<>();

    @Override
    public Optional<StudentUser> get(Integer id) {
        // TODO Auto-generated method stub
        return Optional.empty();
    }
}
```

```

@Override
public int save(StudentUser studnet) {
    int userId = studentUserList.size();
    studnet.setId(userId);
    studentUserList.add(studnet);
    return userId;
}
}

```

```

package com.example.website.domain;

import org.springframework.stereotype.Component;

@Component
public class StudentUser implements User {

    String name;
    String gender;
    String location;
    String college;
    int Id;

    public int getId() {
        return Id;
    }

    public void setId(int id) {
        Id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {

```

```

        this.name = name;
    }
}

```

get Id when we create a user

Persistence layer DAO II

```

import org.springframework.context.annotation.Scope;
import org.springframework.stereotype.Repository;

import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

@Repository
@Scope("singleton")
public class StudentUserDAO implements DAO<StudentUser> {

    private List<StudentUser> studentUserList = new ArrayList<StudentUser>();

    @Override
    public Optional<StudentUser> get(Integer id) {
        if (!studentUserList.isEmpty()) {
            return Optional.of(studentUserList.get(id));
        }
        return Optional.empty();
    }

    @Override
    public int save(StudentUser studnet) {
        int userId = studentUserList.size();

```

```

        studnet.setId(userId);
        studentUserList.add(studnet);
        System.out.println("Student Saved" + userId);
        return userId;
    }
}

```

```

package com.example.website.domain;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;

import com.example.website.repository.StudentUserDAO;

@Component
public class StudentUser implements User {

    @Autowired
    StudentUserDAO studentUserDAO;

    String name;
    String gender;
    String location;
    String college;
    int Id;

    public int getId() {
        return Id;
    }

    public void setId(int id) {
        Id = id;
    }
}

```

```

public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}

public String getGender() {
    return gender;
}

public void setGender(String gender) {
    this.gender = gender;
}

public String getLocation() {
    return location;
}

public void setLocation(String location) {
    this.location = location;
}

public String getCollege() {
    return college;
}

public void setCollege(String college) {
    this.college = college;
}

@Override
public boolean createUser(String name, String gender, String location, String college) {

```

```

        this.name = name;
        this.gender = gender;
        this.location = location;
        this.college = college;
        return true;
    }

    @Override
    public Integer saveUser() {
        System.out.println(this.name);
        return studentUserDAO.save(this);
    }
}

```

```

@Override
public int save(StudentUser studnet) {
    int userId = studentUserList.size();
    studnet.setId(userId);
    studentUserList.add(studnet);
    System.out.println("Student Saved");
    return userId;
}

```



What happens in Java if you don't use the Optional class correctly?

Causes `NullPointerException`



While writing a Java program to process user information. You want to retrieve the user's address from a database. However, the address may

not exist for some users. To handle this problem/exception, what could be used in Java?

Use the Java Optional class, which allows you to handle the case of missing values more elegantly and concisely.



While developing a Java application that stores and manages a collection of objects of various types, such as integers, strings, and custom objects. Which of the following approaches is the most suitable for this task?

Using generics to define a single class or method that can work with objects of any type. Generics in Java allow defining classes, interfaces, and methods that work with objects of any type rather than just a single type, making the code more flexible and usable. A type parameter defined using angle brackets represents the type of tree generic will work with. This type is specified when we're creating an instance or calling the method. Using generics helps prevent runtime errors by catching type-related issues at compile time.